

Evoking Harmony via Convolution

AuthorA¹ and AuthorB²

¹ InstituteA

² InstituteB

`your.email@yourdomain.com`

Abstract. I demonstrate how to impart, or evoke, the tonality of a chord from a source sound while, at the same time, minimizing artifacts and creating an interesting effect. Conceptually, the effect consists of convolving the source signal with the pitch-classes of the chord. I develop alternative implementations of this effect in Csound, analyze the characteristics of each implementation, and demonstrate some musical uses.

Keywords: csound harmony convolution

1 Introduction

When I was a boy my father drove with me several times from our home in Salt Lake City to Los Angeles on business trips. He drove across the desert without stopping. Long stretches of the highway were empty and straight. I lay with my head against the window post and listened to the sound of the car, the road, and the wind. In the rushing noise I heard whistling voices singing counterpoint.

Ever since that time I have wondered if there is some way to produce this effect in electroacoustic music. I have now performed some experiments using Csound [1], and here I report my results. These include some user-defined op-codes, a Csound test orchestra, some input soundfiles, the resulting test soundfile, and an example piece demonstrating some musical uses. All of these materials are available at my Github repository [2].

In brief, it is possible to approximate what I heard by convolving a source signal with the pitch-classes of a chord. This functions as a filter to emphasize those frequencies of the source that also occur in the chord. I call this effect *chord convolution*.

To convolve with the pitch-classes of a chord means to convolve with an impulse response consisting of all sinusoids within the range of human hearing that belong to any of the pitch-classes of the chord. For example, the chord C major consists of pitch-classes C, E, and G. For the pitch-class C, there is one sinusoid at 32.703 Hz, one at 65.406 Hz, and so on up to 16,744.036 Hz; for G, one sinusoid at 41.203 Hz and so on up to 21,906.164 Hz; and for E, one at 48.999 Hz and so on up to 12,543.854 Hz. Please note, pitch-classes are not limited to 12-tone equal temperament; *pitch-class* is simply an equivalence class that identifies frequencies *modulo* the octave.

Table 1. Chord-convolution UDOs in Csound.

UDO	Basis	Main control
<code>ImpartChordWithFilterbank</code>	<code>resonz</code> [4]	bandwidth (cents)
<code>ImpartChordWithFtconv</code>	<code>ftconv</code> [5]	impulse width (cycles)
<code>ImpartChordWithMultiFtconv</code>	per-partial <code>ftconv</code>	width (cycles per partial)

The main artifact of concern in this case is the smearing caused by convolution. Because in all representations of sound there is an irreducible Gabor uncertainty of information, a smallest possible area of time $\times$ frequency which implies an indeterminacy relation between time and frequency [3], this artifact cannot be eliminated, but it can be minimized. We hear it as ringing (smearing of time) or blurring of pitch (smearing of frequency). The chord impulse response can be made shorter to increase time resolution at the cost of frequency resolution, or it can be made longer to increase frequency resolution at the cost of time resolution. The shortest possible impulse response consists of one cycle of the fundamental frequency of the chord. The best value of this tradeoff may be different for different source signals and for different musical purposes, so it is made into a parameter for the effect — the duration of the chord impulse response.

Human factors impose other fundamental limitations. People cannot identify pitch within much less than 50 milliseconds, and features of interest in the source signal, if not sparser than the convolution ring, will be smeared together at the chord frequencies, which may or may not be desirable.

A further optimization consists of convolving with a separate impulse response for each component sinusoid of the chord’s pitch-classes, making the impulse shorter as the frequency increases. This is a form of multi-resolution signal processing. As a result, the smearing is larger at lower frequencies and smaller at higher frequencies. This not only significantly reduces the ringing artifact, but also increases the subjective time resolution of the effect.

2 Methods

Direct convolution with N cycles of each chord partial is optimal but impractical; I did not implement it. Three efficient variants are compared (Table 1). All accept up to six chord voices (MIDI key numbers, possibly fractional), wet/dry mix, and output gain; full signatures are in the repository [2].

The filter bank uses one IIR bandpass per partial; narrow bandwidth increases frequency resolution and ringing. Partitioned convolution uses a single finite impulse response per partial. Multi-`ftconv` applies a separate partitioned convolution at each partial, combining multi-resolution smearing with reasonable CPU cost.

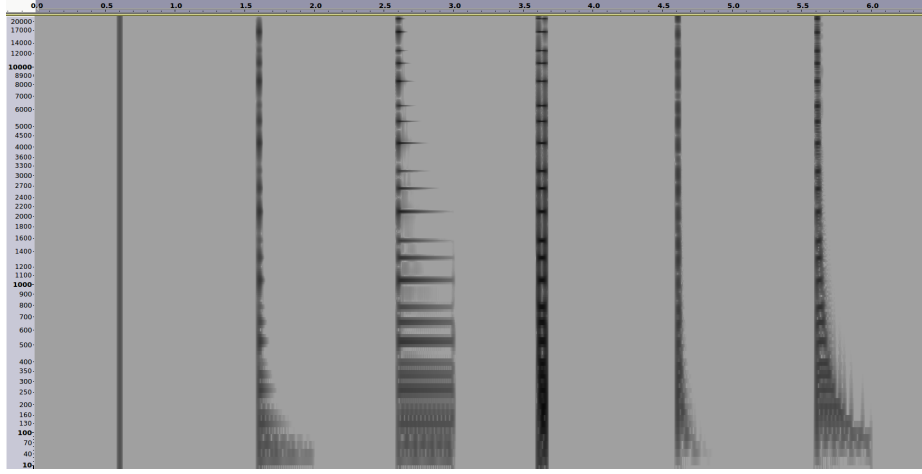


Fig. 1. Impulse responses (narrow analysis window). Columns: unprocessed; filter bank 100 cents; 5 cents; **ftconv** (1 cycle); multi-**ftconv** 8 cycles; 32 cycles.

3 Results

Four source signals probe the implementations: (1) unit impulse; (2) rain (noisy, unpitched); (3) struck steel bowl (pitched, inharmonic); (4) rainspout with radio (mixed time scales). Each was processed with six settings: filter bank at 100 and 5 cents; partitioned convolution at one fundamental cycle; multi-resolution convolution at 8 and 32 cycles per partial. A test soundfile cycles through all combinations; a separate piece applies the effect to field recordings.

Figure 1 summarizes impulse responses (narrow spectrogram window). Single-resolution **ftconv** beats the filter bank on time resolution; multi-**ftconv** is sharpest at high frequencies. Wider settings (5 cents, 32 cycles) lengthen ringing, as expected.

Figure 2 shows rain (representative unpitched source; bowl and rainspout behave similarly—harmonic shadow on pitched sources, arpeggiated “dancing” with a wet filter bank on sparse material). Narrow filter bands do not always sound *more* tonal: on rain, 100 cents sounds stronger than 5 cents because resonances overlap; on the bowl, narrow bands often fail to catch inharmonic partials. Multi-**ftconv** at 8 cycles balances soft tonality with sharper attacks than plain **ftconv**. See [6] for more results.

A sound walk with a field recorder at twilight alongside a ruined castle in a French village demonstrates various applications of harmonic evocation to ambient sounds [7]

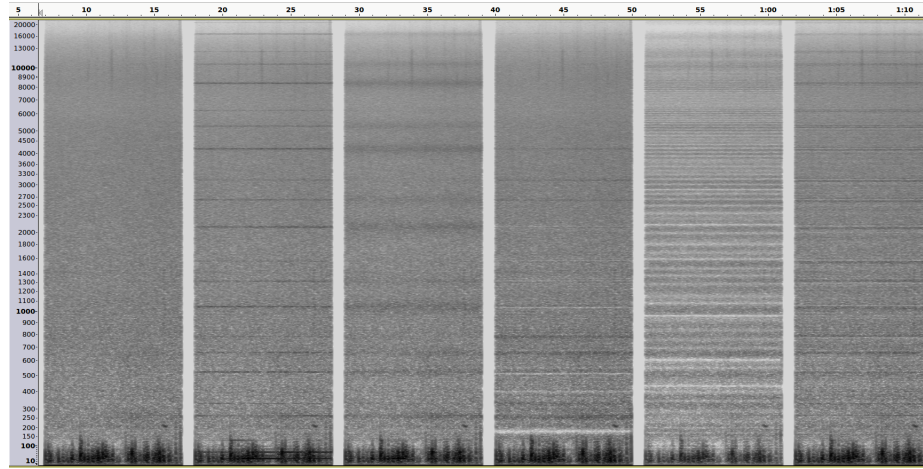


Fig. 2. Rain: same six processing settings as Fig. 1.

4 Discussion

Chord convolution resembles comb filtering or a channel vocoder, but chord tonality is audible when time/frequency settings match the source. It works best when harmonic sensation is subtle—on unpitched or varied textures, or as a “harmonic shadow” on pitched sources in a different key. A wetter, longer smear (filter bank) can suggest ringing arpeggios on rhythmic material. Implementation and parameter choices are audible but nuanced; supplementary figures and audio on GitHub cover bowl, rainspout, and time-domain responses.

Note. This abbreviated version was edited for length with assistance from generative AI tools; the author reviewed the text and retains responsibility for all technical content.

References

1. Barry Vercoe, John ffitch, et al. Csound home page, Accessed 3 July 2026. <http://csound.com>.
2. Michael Gogins. Evoking harmony via convolution (materials). <https://github.com/gogins/michael.gogins.studio/tree/main/music/ICSC-2026>, 2026. GitHub repository. Accessed 3 July 2026.
3. Dennis Gabor. Theory of communication. *Journal of the Institution of Electrical Engineers*, 93(26):429–441, 1946.
4. Csound Developers. **resonz** — resonant bandpass filter. <https://csound.com/docs/manual/resonz.html>. Csound Reference Manual. Accessed 3 July 2026.
5. Csound Developers. **ftconv** — convolution using the fast fourier transform. <https://csound.com/docs/manual/ftconv.html>. Csound Reference Manual. Accessed 3 July 2026.

6. Michael Gogins. Evoking harmony test results. <https://github.com/gogins/michael.gogins.studio/tree/main/music/ICSC-2026>, 2026. GitHub repository. Accessed 3 July 2026.
7. Michael Gogins. Evoking harmony sound walk. <https://github.com/gogins/michael.gogins.studio/tree/main/music/ICSC-2026>, 2026. GitHub repository. Accessed 3 July 2026.